

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
SOLID Principles



Disusun Oleh:

Alifa Fatin Ramadhani 105224019

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

A. Konsep SOLID Principles

SOLID Principles adalah lima prinsip desain dalam Pemrograman Berorientasi Objek (OOP) yang diperkenalkan oleh Robert C. Martin (Uncle Bob). Prinsip ini bertujuan untuk membuat perangkat lunak menjadi lebih mudah dipahami, fleksibel, scalable (mudah dikembangkan), dan maintainable (mudah dirawat). Dengan menerapkan SOLID, kode menjadi tidak kaku terhadap perubahan, sehingga ketika ada penambahan fitur baru (misalnya jalur mahasiswa baru atau migrasi database), kita tidak perlu membongkar kelas inti yang berpotensi merusak fitur lama.

B. Konsep Utama ESolid Principles

1. S - Single Responsibility Principle (SRP)

Sebuah kelas hanya boleh memiliki satu alasan untuk berubah, artinya satu kelas hanya memiliki satu tanggung jawab. Jika sebuah kelas mengurus banyak hal (seperti validasi, cetak PDF, dan koneksi database sekaligus), maka perubahan kecil pada satu fitur bisa merusak fitur lainnya.

2. O - Open/Closed Principle (OCP)

Entitas perangkat lunak harus terbuka untuk ekstensi (penambahan fitur baru) tetapi tertutup untuk modifikasi (tidak perlu mengubah kode yang sudah ada). Ini biasanya diimplementasikan dengan menggunakan interface dan polymorphism.

3. L - Liskov Substitution Principle (LSP)

Objek dari kelas turunan harus bisa menggantikan objek kelas induknya tanpa merusak kebenaran aplikasi. Artinya, subclass harus bisa berdiri sebagai pengganti superclass tanpa menimbulkan error atau perilaku yang tidak terduga.

4. I - Interface Segregation Principle (ISP)

Klien tidak boleh dipaksa bergantung pada method yang tidak mereka gunakan. Lebih baik memiliki banyak interface kecil dan spesifik daripada satu interface besar yang memaksa kelas mengimplementasikan method yang tidak mereka butuhkan (yang biasanya berakhir dengan UnsupportedOperationException).

5. D - Dependency Inversion Principle (DIP)

Modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah. Keduanya harus bergantung pada abstraksi (interface). Dengan begitu, kita bisa mengganti implementasi (misalnya dari MySQL ke Cloud NoSQL) tanpa harus mengubah kode inti.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
Class cloudNoSQL			
1			Class implementasi database untuk cloud no SQL
Class konekMYSQL			
1			Class implementasi database untuk MYSQL
Class Database			
		interface	Sebuah abtrsk untuk koneksi database agar tidak terikat satu jenis DB
Class MatakulPraktikums			
1	nama	String	Menyimpan nama mata kuliah yang ada di praktikum (yang membutuhkan asisten dan alat lab)
2	sks	int	Menyimpan jumlah sks
Class MatkulAspraImpl			
1	nama	String	Menyimpan mata kuliah yang butuh praktikum
2	sks	int	Menyimpan jumlah SKS
Class MatkulKKN			
1	nama	String	Menyimpan mata kuliah KKN
2	sks	int	Menyimpan jumlah SKS
Class MatkulTeori			
1	nama	String	Menyimpan mata kuliah teori
2	sks	int	Menyimpan jumlah SKS
Class Matkul			
1		interface	Interface dasar untuk semua mata kuliah (ISP dan LSP)
Class MatkulAsprak			

1		interface	Interface spesifik untuk matkul yang butuh asisten (ISP)
Class MatkulPraktikum			
1		Interface	Interface spesifik untuk matkul yang butuh alat lab (ISP)
Class Mahasiswa			
1	nama	String	Menyimpan nama mahasiswa
2	StrategiUKT	HitungUKT	Menyimpan objek strategi perhitungan UKT (OCP)
Class RepositoriKRS			
1	db	Database	Menyimpan objek koneksi database
Class SistemKRSMAnager			
1	validator	ValidatorKRS	Objek untuk validasi prasyarat (SRP)
2	generator	GeneratorDokumenKRS	Objek untuk cetak PDF (SRP)
3	repository	RepositoriKRS	Objek untuk simpan ke DB (SRP & DIP)
Class Main			
1	matakuliahList	matkul[]	Array daftar mata kuliah yang diambil
2	manager	SistemKRSMAnager	Managr untuk mengatur sistem KRS

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1.	<pre>public class coludNoSQL()</pre>	Constructor	Inisialisasi koneksi Cloud NoSQL
2	<pre>koneksiDatabase ()</pre>	Procedural	Menghubungkan ke Cloud NoSQL
3	<pre>public void simpanData(String data)</pre>	Procedural	Menyimpan data ke Cloud NoSQL
4	<pre>konekMySQL()</pre>	Constructor	Inisialisasi koneksi MySQL
5	<pre>public MatakulPraktikums(String nama, int sks)</pre>	Constructor	Inisialisasi nama dan SKS matkul praktikum+asprak
6	<pre>public void asistenPraktikum()</pre>	Procedural	Menugaskan asisten dosen (dari MatkulAsprak)
7	<pre>public void cekAlatLab()</pre>	Procedural	Mengecek alat lab (dari MatkulPraktikum)
8	<pre>public Matakuliah(String nama, int sks)</pre>	Constructor	Inisialisasi nama dan SKS matkul asprak
9	<pre>public MatkulKKN(String nama, int sks)</pre>	Constructor	Inisialisasi nama dan SKS matkul KKN
10	<pre>public MatkulTeori(String nama, int sks)</pre>	Constructor	Inisialisasi nama dan SKS matkul teo

11	<pre>public Mahasiswa(String nama, HitungUKT strategiUKT)</pre>	Constructor	Inisialisasi nama dan SKS matkul teo
12	<pre>Mahasiswa(String g, HitungUKT)</pre>	Procedural	Mendelegasikan hitung UKT ke strategi yang di-inject
13	<pre>getNama()</pre>	Getter	Mengambil nama mahasiswa
14	<pre>RepositoriKRS(Database)</pre>	Constructor	Injeksi dependency database (DIP)
15	<pre>public void simpanRiwayatKRS(String dataKRS)</pre>	Procedural	Menyimpan data ke database melalui abstraksi
16	<pre>public SistemKRSManger(ValidatorKRS validator, GeneratorDokumenKRS generator, RepositoriKRS repository)</pre>	Constructor	Injeksi semua dependency
17	<pre>public void prosesPengisianKRS(Mahasiswa mhs, Matkul[] matakuliahList)</pre>	Procedural	alur KRs dan validasi simpan
18	<pre>public boolean validasiSyarat(Mahasiswa mhs, Matkul mk)</pre>	Procedural	Memvalidasi persyaratan mata kuliah
19	<pre>public void cetakDraftKRS(String namaMahasiswa,</pre>	Procedural	Mencetak draf krs

	<code>double totalUKT)</code>		
--	-----------------------------------	--	--

IV. Dokumentasi dan Pembahasan Code

A. Database.java dan Implementasinya (coludNoSQL.java, konekMySQL.java)

Database.java

```
public interface Database { //menerapkan DEPENDENCY
INVERSION PRINCIPLE (DIP)
    void koneksiDatabase();
    void simpanData(String data);
}
```

coludNoSQL.java

```
public class coludNoSQL implements Database{ //menerapkan
DEPENDENCY INVERSION PRINCIPLE (DIP)
    @Override
    public void koneksiDatabase(){
        System.out.println("Koneksi ke Cloud NoSQL
berhasil!");
    }

    @Override
    public void simpanData(String data){
        System.out.println("Menyimpan data ke Cloud NoSQL:
" + data);
    }
}
```

konekMySQL.java

```
public class konekMySQL implements Database { //menerapkan
DEPENDENCY INVERSION PRINCIPLE (DIP)
    @Override
    public void koneksiDatabase() {
        System.out.println("Koneksi ke MySQL berhasil!");
    }
}
```

```

    }

    @Override
    public void simpanData(String data) {
        System.out.println("Menyimpan data ke MySQL: " +
data);
    }
}

```

- Class database merupakan interface yang menjadi abstraksi untuk koneksi database.
- Menerapkan DIP (Dependency Inversion Principle) karena kelas tingkat tinggi (RepositoryKRS) tidak bergantung pada implementasi konkret (MySQL/NoSQL), melainkan pada abstraksi
- • coludNoSQL dan konekMYSQL adalah kelas konkret yang mengimplementasikan interface Database
- • Jika rektorat ingin migrasi dari MySQL ke Cloud NoSQL, cukup ganti satu baris di Main.java dari new konekMYSQL() menjadi new coludNoSQL(), tanpa menyentuh kode inti
- • Ini menjawab masalah pada soal di mana tim IT lumpuh karena sistem terikat pada MySQL

B. Matkul.java, MatkulAsprak.java, MatkulPraktikum.java (Interface)

Matkul.java

```

public interface Matkul { //menerapkan ISP dan LSP
    String namaMatkul();
    int jumlahSKS();
}

```

MatkulAsprak.java

```

public interface MatkulAsprak extends Matkul{ //menerapkan
ISP dan LSP
    void asistenPraktikum();
}

```

MatkulPraktikum.java

```

public class MatakulPraktikums implements MatkulAsprak,
MatkulPraktikum {

    private String nama;
}

```



```

private int sks;

public MatakulPraktikums(String nama, int sks) {
    this.nama = nama;
    this.sks = sks;
}

@Override
public void asistenPraktikum() {
    System.out.println("Menugaskan Asisten Dosen untuk
lab....");
}

@Override
public void cekAlatLab() {
    System.out.println("Mengecek alat lab....");
}

@Override
public int jumlahSKS() {
    return sks;
}

@Override
public String namaMatkul() {
    return nama;
}
}

```

- Matkul adalah interface dasar yang wajib dimiliki semua mata kuliah (nama dan SKS)
- MatkulAsprak dan MatkulPraktikum adalah interface spesifik yang meng-extend Matkul
- Menerapkan ISP (Interface Segregation Principle) karena interface dipecah menjadi kecil dan spesifik, sehingga kelas tidak dipaksa mengimplementasikan method yang tidak mereka butuhkan
- Menerapkan LSP (Liskov Substitution Principle) karena kelas MatkulTeori dan MatkulKKN tidak lagi dipaksa mengimplementasikan asistenPraktikum() atau cekAlatLab() yang bisa menyebabkan UnsupportedOperationException
- Ini menjawab masalah pada soal di mana programmer terdahulu membuat interface raksasa yang memicu error saat iterasi data

C. MatakulPraktikums.java, MatkulAsprakImpl.java, MatkulKKN.java, MatkulTeori.java

MatakulPraktikums.java

```
public class MatakulPraktikums implements MatkulAsprak,
MatkulPraktikum {

    private String nama;
    private int sks;

    public MatakulPraktikums(String nama, int sks) {
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public void asistenPraktikum() {
        System.out.println("Menugaskan Asisten Dosen untuk
lab....");
    }

    @Override
    public void cekAlatLab() {
        System.out.println("Mengecek alat lab....");
    }

    @Override
    public int jumlahSKS() {
        return sks;
    }

    @Override
    public String namaMatkul() {
        return nama;
    }

}
```

MatkulAsprakImpl.java

```
public class MatkulAsprakImpl implements MatkulAsprak {
    private String nama;
    private int sks;

    public MatkulAsprakImpl(String nama, int sks) {
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public String namaMatkul() {
        return nama;
    }

    @Override
    public int jumlahSKS() {
        return sks;
    }

    @Override
    public void asistenPraktikum() {
        System.out.println("Menyiapkan asisten praktikum
untuk mata kuliah " + nama);
    }
}
```

MatkulKKN.java

```
public class MatkulKKN implements Matkul{

    private String nama;
    private int sks;

    public MatkulKKN(String nama, int sks) {
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public int jumlahSKS() {
        return sks;
    }
}
```

```

    }

    @Override
    public String namaMatkul() {
        return nama;
    }
}

```

- MatakulPraktikums mengimplementasikan dua interface sekaligus (MatkulAsprak dan MatkulPraktikum) karena mata kuliah ini butuh asisten sekaligus alat lab
- MatkulAsprakImpl hanya mengimplementasikan MatkulAsprak karena hanya butuh asisten
- MatkulKKN dan MatkulTeori hanya mengimplementasikan interface dasar Matkul, sehingga tidak dibebani method yang tidak mereka butuhkan
- Ini adalah wujud nyata dari ISP — setiap kelas hanya "mengambil" perilaku yang memang sesuai dengan dirinya
- Tidak ada lagi throw new UnsupportedOperationException() yang merusak aplikasi

D. HitungUKT.java, UKTReg.java, UKTBidikmisi.java, UKTBKM.java

HitungUKT.java

```

public interface HitungUKT { //menggunakan metode OCP
    double hitungUKT(int ambilSKS);
}

```

UKTReg.java

```

public class UKTReg implements HitungUKT{
    @Override
    public double hitungUKT(int ambilSKS) {
        return ambilSKS * 500000;
    }
}

```

UKTBKM.java

```

public class UKTBKM implements HitungUKT {

```

```

@Override
public double hitungUKT(int ambilSKS) {
    return ambilSKS * 300000;
}
}

```

- HitungUKT adalah interface yang mendefinisikan kontrak perhitungan UKT
- Menerapkan OCP (Open/Closed Principle) — sistem terbuka untuk ekstensi (jalur mahasiswa baru) tapi tertutup untuk modifikasi
- Setiap jalur mahasiswa (Reguler, Bidikmisi, MBKM) memiliki kelas sendiri-sendiri dengan rumus masing-masing
- Jika rektorat menambah jalur baru (misalnya jalur Internasional), cukup buat kelas baru UKTInternasional.java tanpa menyentuh kelas yang sudah ada
- Ini menjawab masalah pada soal di mana ratusan baris if-else di SistemKRSManger lama membuat penambahan jalur baru selalu merusak rumus lama

E. Mahasiswa.java

```

public class Mahasiswa {
    private String nama;
    private HitungUKT strategiUKT;

    public Mahasiswa(String nama, HitungUKT strategiUKT) {
        this.nama = nama;
        this.strategiUKT = strategiUKT;
    }

    public double hitungUKT(int jumlahSKS) {
        return strategiUKT.hitungUKT(jumlahSKS);
    }

    public String getNama() {
        return nama;
    }
}

```

- Class Mahasiswa menggunakan Dependency Injection melalui constructor untuk menerima strategi perhitungan UKT
- Menerapkan OCP karena method hitungUKT() mendelegasikan perhitungan ke objek strategiUKT yang di-inject, sehingga tidak perlu if-else di dalam class ini

- Menerapkan DIP karena Mahasiswa bergantung pada abstraksi HitungUKT, bukan pada kelas konkret seperti UKTReg atau UKTBidikmisi
- Getter getNama() digunakan untuk mengakses nama mahasiswa yang nantinya dipanggil di SistemKRSManager

F. ValidatorKRS.java, GeneratorDokumenKRS.java, RepositoriKRS.java

ValidatorKRS.java

```
public class ValidatorKRS {
    public boolean validasiSyarat(Mahasiswa mhs, Matkul
mk) {

        System.out.println("Memvalidasi prasyarat untuk "
+ mk.namaMatkul() + "...");
        return true;
    }
}
```

GeneratorDokumenKRS.java

```
public class GeneratorDokumenKRS {
    public void cetakDraftKRS(String namaMahasiswa, double
totalUKT) {
        System.out.println("--- MENCETAK DRAF KRS PDF
---");
        System.out.println("Kop Surat: UNIVERSITAS
PERTAMINA"); // Jika kop berubah, hanya kelas ini yang
diedit!
        System.out.println("Nama: " + namaMahasiswa);
        System.out.println("Total UKT: Rp " + totalUKT);

        System.out.println("-----");
    }
}
```

RepositoriKRS.java

```
public class RepositoriKRS {
    private Database db;

    public RepositoriKRS(Database db) {
        this.db = db;
    }
}
```

```

public void simpanRiwayatKRS(String dataKRS) {
    db.koneksiDatabase();
    db.simpanData(dataKRS);
}
}

```

- Ketiga class ini adalah wujud dari SRP (Single Responsibility Principle) — setiap class hanya memiliki satu tanggung jawab
- ValidatorKRS hanya bertanggung jawab atas validasi prasyarat
- GeneratorDokumenKRS hanya bertanggung jawab atas pencetakan PDF — jika kop surat berubah, hanya class ini yang diedit, tidak akan merusak logika penyimpanan database
- RepositoriKRS hanya bertanggung jawab atas penyimpanan data ke database
- RepositoriKRS juga menerapkan DIP karena bergantung pada interface Database, bukan pada koneksiMySQL atau koneksiNoSQL secara langsung
- Ini menjawab masalah pada soal di mana mengubah kop surat PDF tanpa sengaja merusak algoritma penyimpanan database

G. SistemKRSMAnager.java

```

public class SistemKRSMAnager {
    private ValidatorKRS validator;
    private GeneratorDokumenKRS generator;
    private RepositoriKRS repository;

    public SistemKRSMAnager(ValidatorKRS validator,
GeneratorDokumenKRS generator, RepositoriKRS repository) {
        this.validator = validator;
        this.generator = generator;
        this.repository = repository;
    }

    public void prosesPengisianKRS(Mahasiswa mhs, Matkul[]
matakuliahList) {
        System.out.println("Mulai proses krs untuk... " +
mhs.getNama().toUpperCase());

        int totalSKS = 0;
        for (Matkul mk : matakuliahList) {
            if(validator.validasiSyarat(mhs, mk)) {

```

```

        System.out.println("Gagal: Prasyarat " +
mk.namaMatkul() + " belum terpenuhi!");
    }

    if (mk instanceof MatkulPraktikum) {
        ((MatkulPraktikum) mk).cekAlatLab();
    }
    totalSKS += mk.jumlahSKS();
}

double totalUKT = mhs.hitungUKT(totalSKS);
generator.cetakDraftKRS(mhs.getNama(), totalUKT);
String dataKRS = "KRS Mahasiswa: " + mhs.getNama()
+ " dengan total SKS: " + totalSKS + " dengan total UKT: "
+ totalUKT;
repository.simpanRiwayatKRS(dataKRS);

System.out.println("Proses KRS Selesai.....");
}
}

```

- SistemKRSManager adalah kelas orkestrator yang hanya bertugas mengatur alur proses KRS, bukan mengerjakan semuanya sendiri
- Menerima semua dependency melalui constructor injection (DIP)
- Menerapkan SRP karena tidak lagi mengganggu logika validasi, cetak, dan penyimpanan database
- Menggunakan instanceof untuk mengecek apakah mata kuliah adalah praktikum, lalu memanggil cekAlatLab() — ini aman karena tidak ada lagi UnsupportedOperationException (LSP & ISP)
- mhs.getNama().toUpperCase() adalah method chaining — getNama() mengembalikan String, lalu toUpperCase() (method bawaan Java) mengubahnya menjadi huruf kapital
- Tidak ada lagi if-else untuk perhitungan UKT karena didelegasikan ke mhs.hitungUKT() (OCP)

H. Main.java

```

public class Main {
    public static void main(String[] args) {
        Database dbKampus = new coludNoSQL();

        ValidatorKRS validator = new ValidatorKRS();
        GeneratorDokumenKRS generator = new
GeneratorDokumenKRS();
    }
}

```



```

        RepositoriKRS repository = new
RepositoriKRS(dbKampus);

        SistemKRSMAnager manager = new
SistemKRSMAnager(validator, generator, repository);

        Matkul[] matakuliahList = new Matkul[] {
            new MatakulPraktikums("Matematika", 3),
            new MatkulAsprakImpl("Fisika", 3),
            new MatkulKKN("KKN Sukses!", 4),
        };

        Mahasiswa tono = new Mahasiswa("Tono Santoso", new
UKTReg());
        manager.prosesPengisianKRS(tono, matakuliahList);

        Mahasiswa endang = new Mahasiswa("Endang Nurhayati",
new UKTBidikmisi());
        manager.prosesPengisianKRS(endang, matakuliahList);

        Mahasiswa regis = new Mahasiswa("Regis Yahyo", new
UKTBKM());
        manager.prosesPengisianKRS(regis, matakuliahList);
    }
}

```

- Main berperan sebagai entry point yang menyusun semua dependency dan menjalankan skenario
- Database dbKampus = new coludNoSQL() membuktikan DIP — jika ingin kembali ke MySQL, cukup ganti menjadi new konekMYSQL() tanpa menyentuh kelas lain
- Array Matkul[] dapat menampung berbagai jenis mata kuliah (praktikum, asprak, KKN) secara polimorfik berkat LSP
- Penambahan jalur MBKM (UKTBKM) dilakukan tanpa mengubah satu baris pun di SistemKRSMAnager — ini adalah wujud OCP
- Setiap mahasiswa di-inject dengan strategi UKT yang berbeda melalui constructor, membuktikan OCP dan DIP bekerja harmonis

I. Output dari Program

```

Mulai proses krs untuk... TONO SANTOSO
Memvalidasi prasyarat untuk Matematika...
Gagal: Prasyarat Matematika belum terpenuhi!
Mengecek alat lab....

```

Memvalidasi prasyarat untuk Fisika...
Gagal: Prasyarat Fisika belum terpenuhi!
Memvalidasi prasyarat untuk KKN Sukses!...
Gagal: Prasyarat KKN Sukses! belum terpenuhi!
--- MENCETAK DRAF KRS PDF ---
Kop Surat: UNIVERSITAS PERTAMINA
Nama: Tono Santoso
Total UKT: Rp 5000000.0

Koneksi ke Cloud NoSQL berhasil!
Menyimpan data ke Cloud NoSQL: KRS Mahasiswa: Tono Santoso dengan total SKS: 10 dengan total UKT: 5000000.0
Proses KRS Selesai.....
Mulai proses krs untuk... ENDANG NURHAYATI
Memvalidasi prasyarat untuk Matematika...
Gagal: Prasyarat Matematika belum terpenuhi!
Mengecek alat lab....
Memvalidasi prasyarat untuk Fisika...
Gagal: Prasyarat Fisika belum terpenuhi!
Memvalidasi prasyarat untuk KKN Sukses!...
Gagal: Prasyarat KKN Sukses! belum terpenuhi!
--- MENCETAK DRAF KRS PDF ---
Kop Surat: UNIVERSITAS PERTAMINA
Nama: Endang Nurhayati
Total UKT: Rp 0.0

Koneksi ke Cloud NoSQL berhasil!
Menyimpan data ke Cloud NoSQL: KRS Mahasiswa: Endang Nurhayati dengan total SKS: 10 dengan total UKT: 0.0
Proses KRS Selesai.....
Mulai proses krs untuk... REGIS YAHYO
Memvalidasi prasyarat untuk Matematika...
Gagal: Prasyarat Matematika belum terpenuhi!
Mengecek alat lab....
Memvalidasi prasyarat untuk Fisika...
Gagal: Prasyarat Fisika belum terpenuhi!
Memvalidasi prasyarat untuk KKN Sukses!...
Gagal: Prasyarat KKN Sukses! belum terpenuhi!
--- MENCETAK DRAF KRS PDF ---
Kop Surat: UNIVERSITAS PERTAMINA
Nama: Regis Yahyo
Total UKT: Rp 3000000.0

Koneksi ke Cloud NoSQL berhasil!
Menyimpan data ke Cloud NoSQL: KRS Mahasiswa: Regis Yahyo dengan total SKS: 10 dengan total UKT: 3000000.0
Proses KRS Selesai.....

V. Kesimpulan

Berdasarkan hasil refactoring arsitektur SIAKAD menggunakan prinsip SOLID, sistem yang awalnya merupakan "kelas raksasa" bernama SistemKRSManager telah berhasil dipecah menjadi beberapa kelas modular yang masing-masing memiliki tanggung jawab tunggal. Penerapan Single Responsibility Principle (SRP) memungkinkan perubahan pada satu fitur, seperti modifikasi format kop surat PDF pada GeneratorDokumenKRS, tidak lagi berisiko merusak fitur lain seperti algoritma penyimpanan database di RepositoriKRS. Setiap kelas kini memiliki alasan yang jelas dan spesifik untuk berubah, sehingga kode menjadi lebih mudah dirawat dan dipahami oleh pengembang.

Selain itu, penerapan Open/Closed Principle (OCP) melalui penggunaan interface HitungUKT beserta kelas-kelas implementasinya (UKTReg, UKTBidikmisi, UKTBKM) membuat sistem terbuka untuk penambahan jalur mahasiswa baru tanpa perlu memodifikasi kelas inti. Ketika rektorat menambahkan skema MBKM, pengembang cukup membuat kelas baru tanpa menyentuh rumus UKT yang sudah ada, sehingga menghilangkan risiko merusak perhitungan untuk mahasiswa lama yang sebelumnya dipenuhi oleh ratusan baris if-else. Di sisi lain, Interface Segregation Principle (ISP) dan Liskov Substitution Principle (LSP) diterapkan dengan memecah interface Matkul menjadi interface-interface spesifik seperti MatkulAsprak dan MatkulPraktikum, sehingga kelas MatkulTeori dan MatkulKKN tidak lagi dipaksa mengimplementasikan method yang tidak mereka butuhkan. Hal ini menghilangkan potensi error UnsupportedOperationException yang sebelumnya sering memicu crash saat sistem melakukan iterasi data mata kuliah.

Terakhir, penerapan Dependency Inversion Principle (DIP) melalui interface Database memungkinkan kelas RepositoriKRS tidak lagi terikat secara kaku pada implementasi MySQL. Migrasi ke infrastruktur Cloud NoSQL kini hanya membutuhkan perubahan satu baris kode di kelas Main dari new konekMYSQL() menjadi new coludNoSQL(), tanpa perlu membongkar logika bisnis inti. Dengan kelima prinsip SOLID yang telah diterapkan secara harmonis, sistem SIAKAD kini telah bertransformasi dari arsitektur yang rapuh dan kaku menjadi sistem yang tangkas, scalable, dan siap mengakomodasi inovasi akademik kampus di masa depan tanpa risiko merusak fitur yang sudah berjalan stabil.

VI. Daftar Pustaka

-